



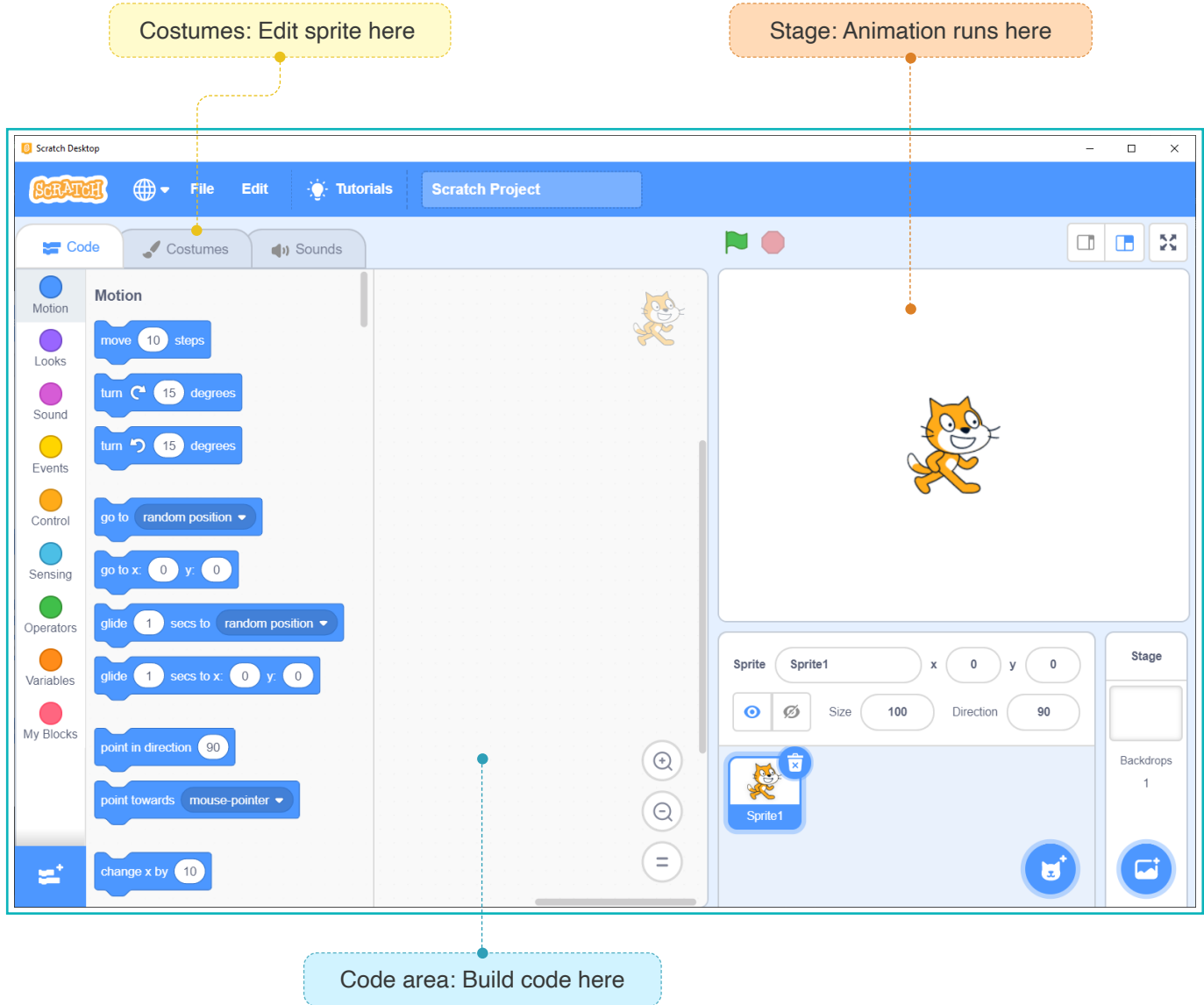
A game is like a computer software. It gives instructions in a sequence. Do you want to learn how to make your own game?

Uncle, how do people make computer games?

Recall:

- **Scratch:** Scratch is a software that uses blocks in a particular order to give instructions. The instructions can be used to create an animation, story, or game. Sprites and Codes are used as operating tools in Scratch.
- **Sprite:** A sprite can be an animated character, a player in a game, or any other object.
- **Code:** Code is a set of instructions given to a computer to perform an action. Scratch uses a block-based code.

5. Conditions



In this lesson, we will use sprites to create a maze game.

What is a maze game?

In a maze game, the player has to find the correct path, out of multiple paths, to reach the goal. There are four main parts of a maze game:

Start point: The point at which the player starts to play.

End point: The goal where the player needs to reach to win.

Path: A way through which the player may reach the goal.

Hurdles: Obstacles in the path to restrict the player on the way to the goal.

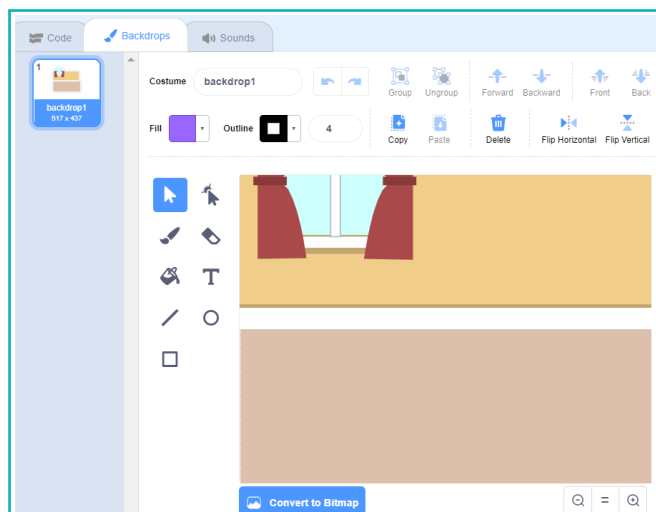
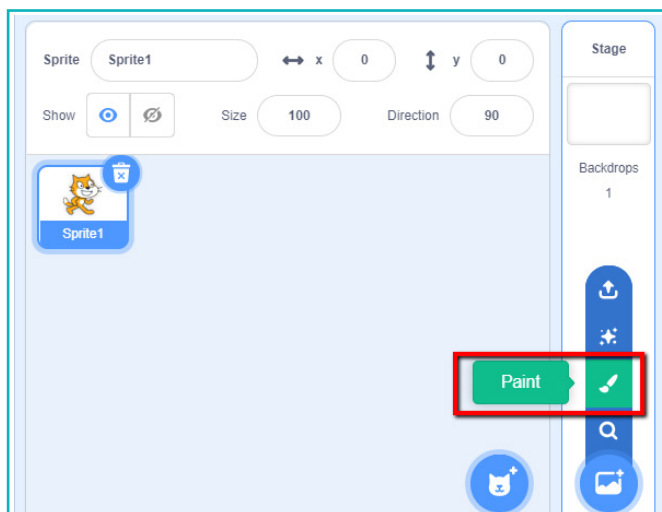
This is what a maze game looks like:



We will create a simple maze game. To do this, we need an appropriate background. We can select an image that is available in the backdrop library or create our own.

Steps

1 Create a background

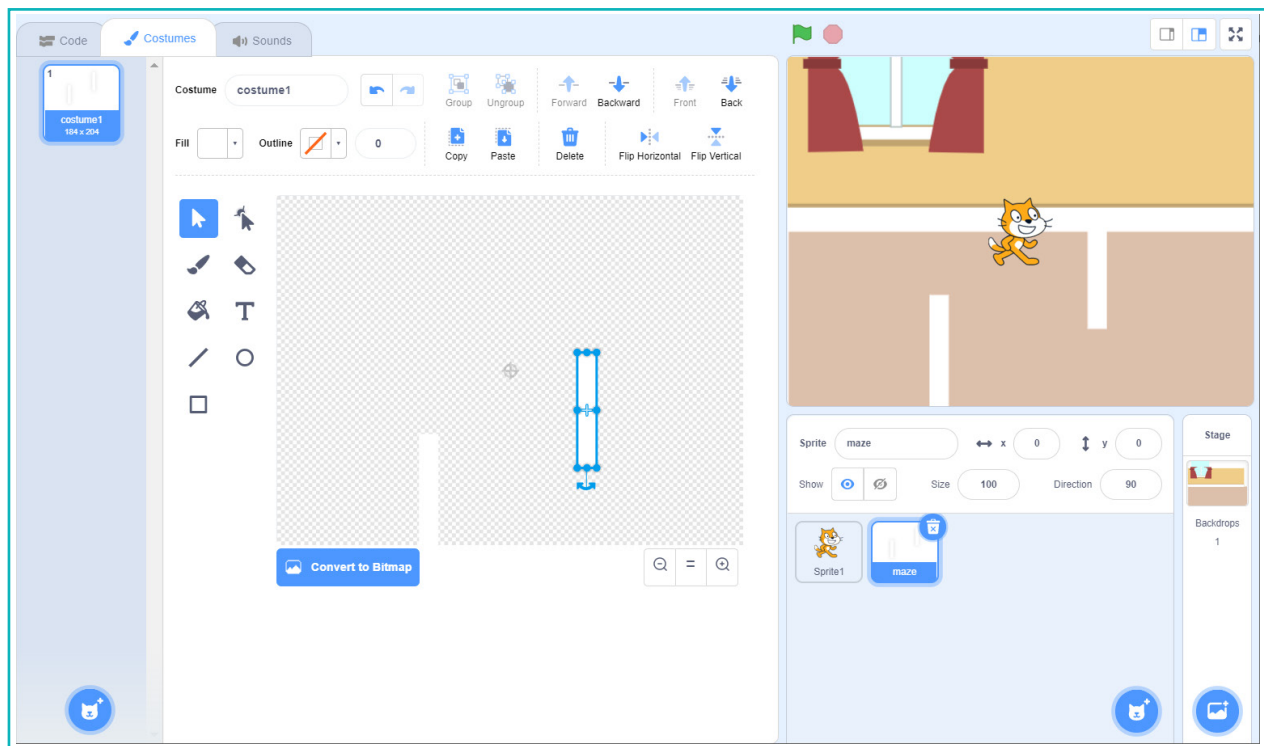


Steps

2 Create Hurdles



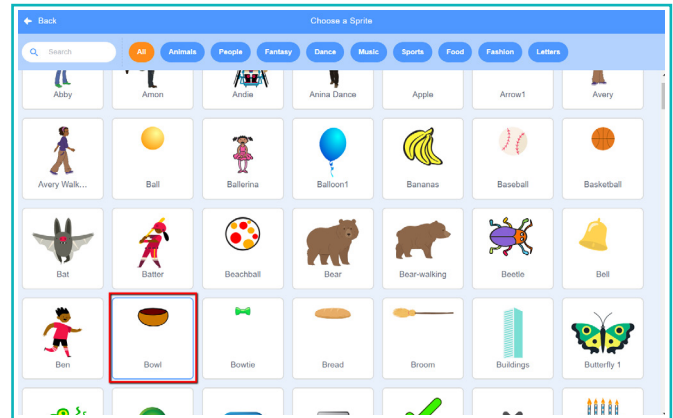
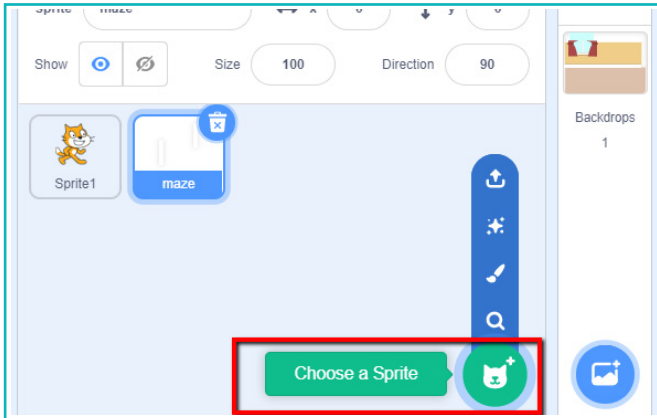
The background is plain right now. To make it look like a maze, we need to add hurdles. We will create white pillars as the hurdles.



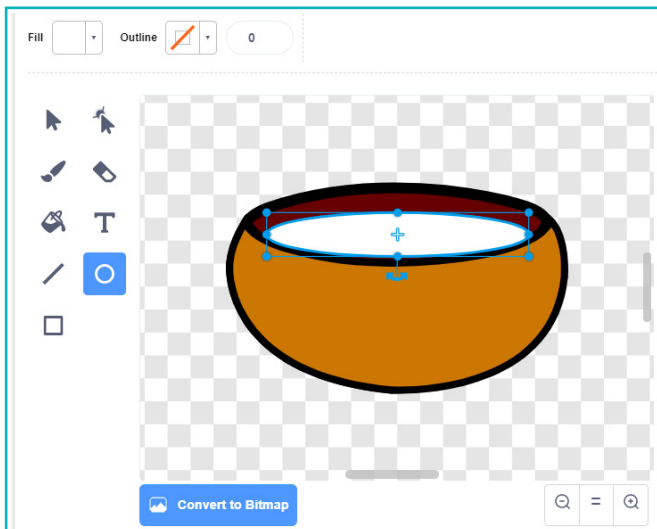
With hurdles, our path is ready, but it is incomplete without a player and an end goal. Tom, the cat, is at the centre of the stage by default. We will place him at the start position. For the goal, we will create a milk bowl.

Steps

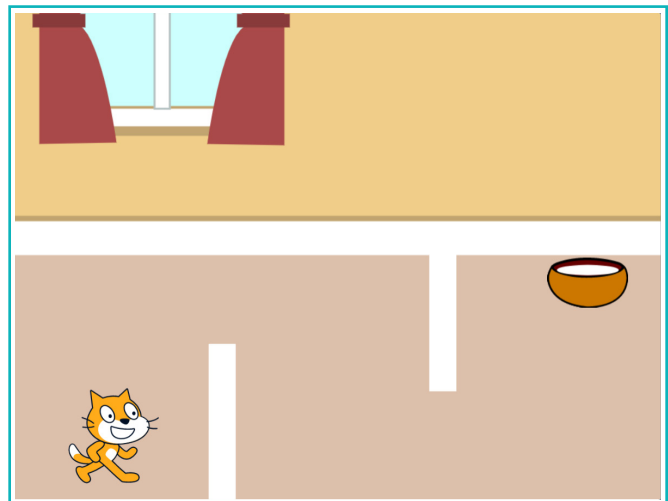
3 Create an end point goal



Edit the bowl sprite so that it is full of milk.



Drag and drop Tom at the start position.
The positions of the sprites will look like this:



Now our game has all the elements of a maze game.

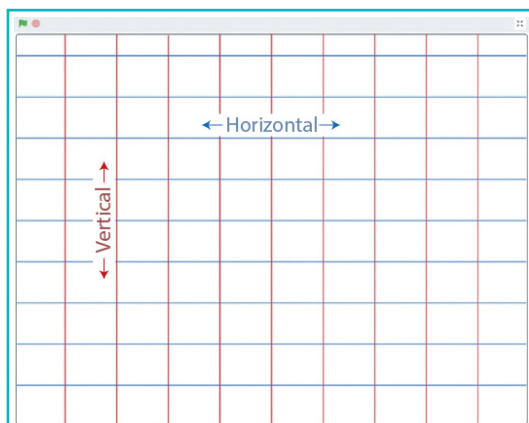
Exercise

State whether true or false

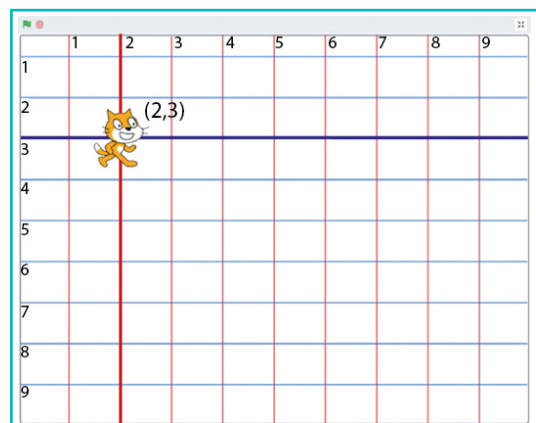
- 1 The costumes tab allows users to change the look of a sprite.
 ▲ _____
- 2 The block palette contains all the code blocks.
 ▲ _____
- 3 The stage area contains all the sprites that are used in the animation/games.
 ▲ _____

A player must begin at the start point when the game starts. Tom must remain at the start point of our game. When we restart the game, Tom will move to the centre by default. We will fix his position at the start point.

First we have to understand how Scratch decides the position of any sprite. Scratch uses horizontal (sleeping) and vertical (standing) lines to understand object positions.



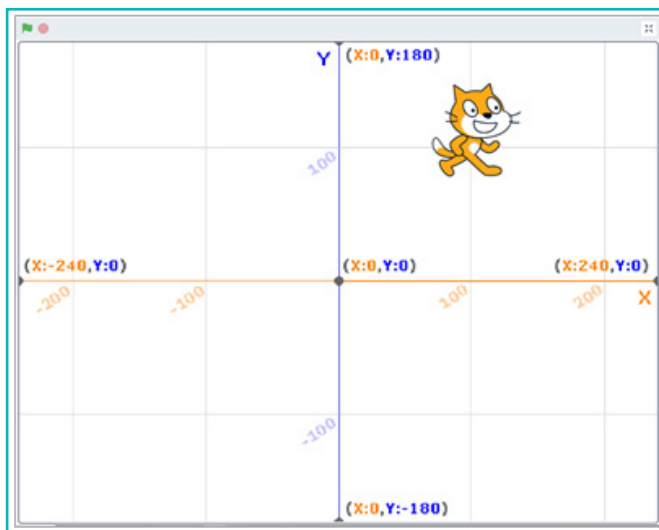
The structure made with horizontal and vertical lines is called a grid. If we place Tom on this grid, then scratch can identify his position. In the following image, the position of Tom is (2,3).



5. Conditions

We label the dark central horizontal line as 'X' and the dark central vertical line as 'Y'. The point where X and Y meet each other is the centre, represented as (0,0). As we move right from the centre, the value of X increases. As we move left from the centre, the value of X decreases. Similarly, the value of Y increase above the centre and decreases below the centre. Here, the position of Tom is (2X, 3Y).

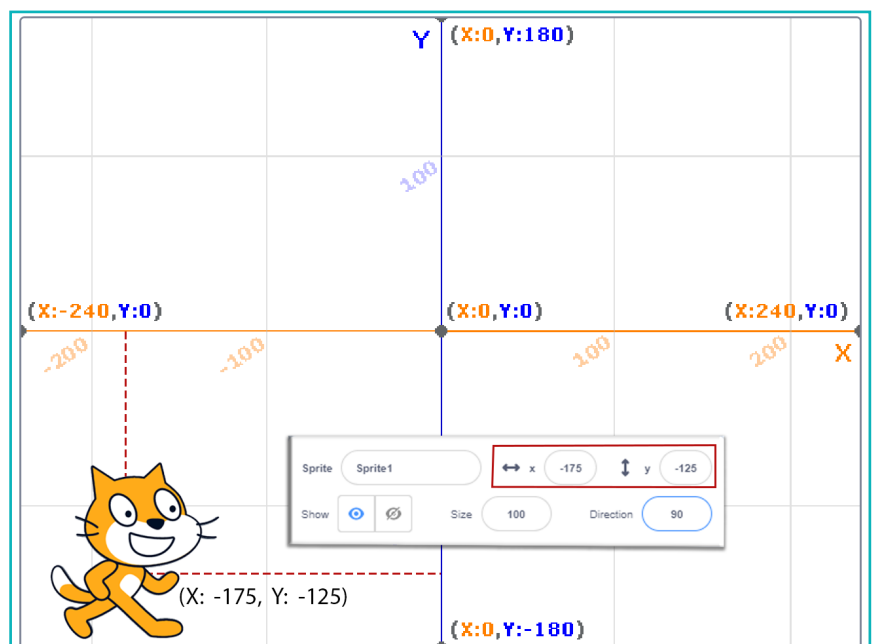
Scratch uses the same method to fix the position of a sprite. It divides the stage into grids and identifies the sprite's position. Observe the following image:



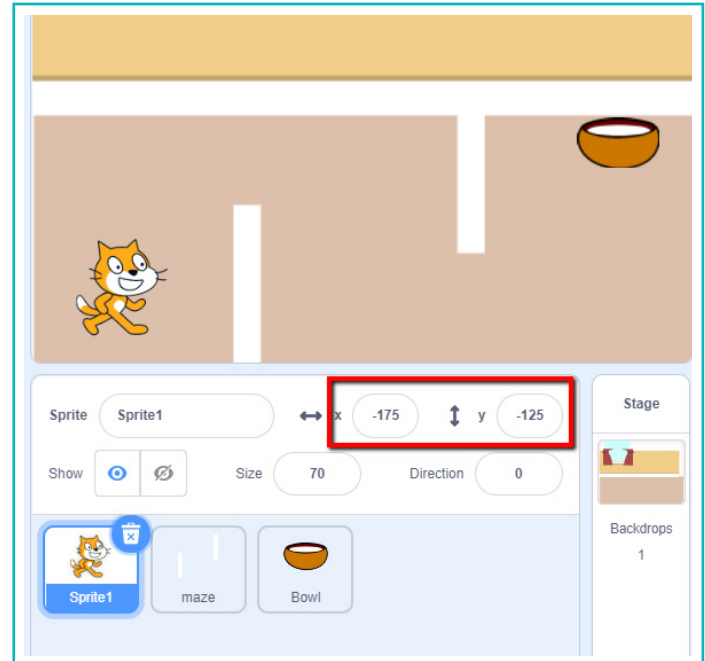
In this figure, there are multiple smaller grids. So, the position of Tom is (100, 100).



Observe the following image. We will use (-175, -125) as the position for Tom in our game.

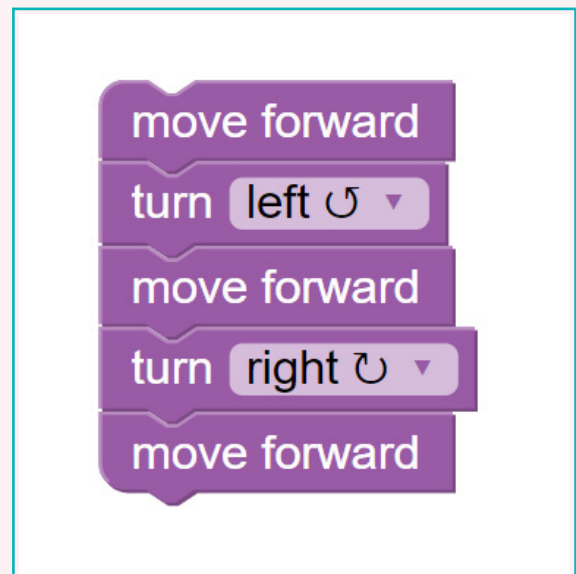
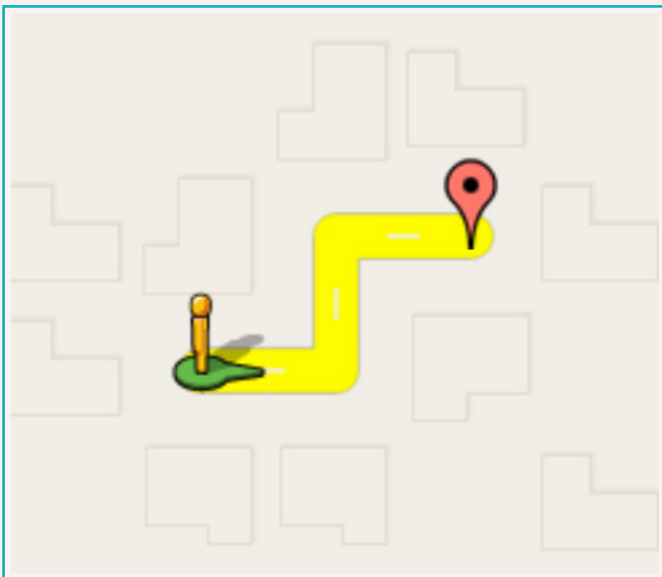


We want to fix Tom's position. To do this, we will create a code.



What is a code?

A computer code is a set of instructions. The computer follows the instructions to complete a task. A code is the step-by-step instructions that tell the computer what to do. When we write these instructions, it is called coding. We can apply specific codes to the sprite and create any kind of game.

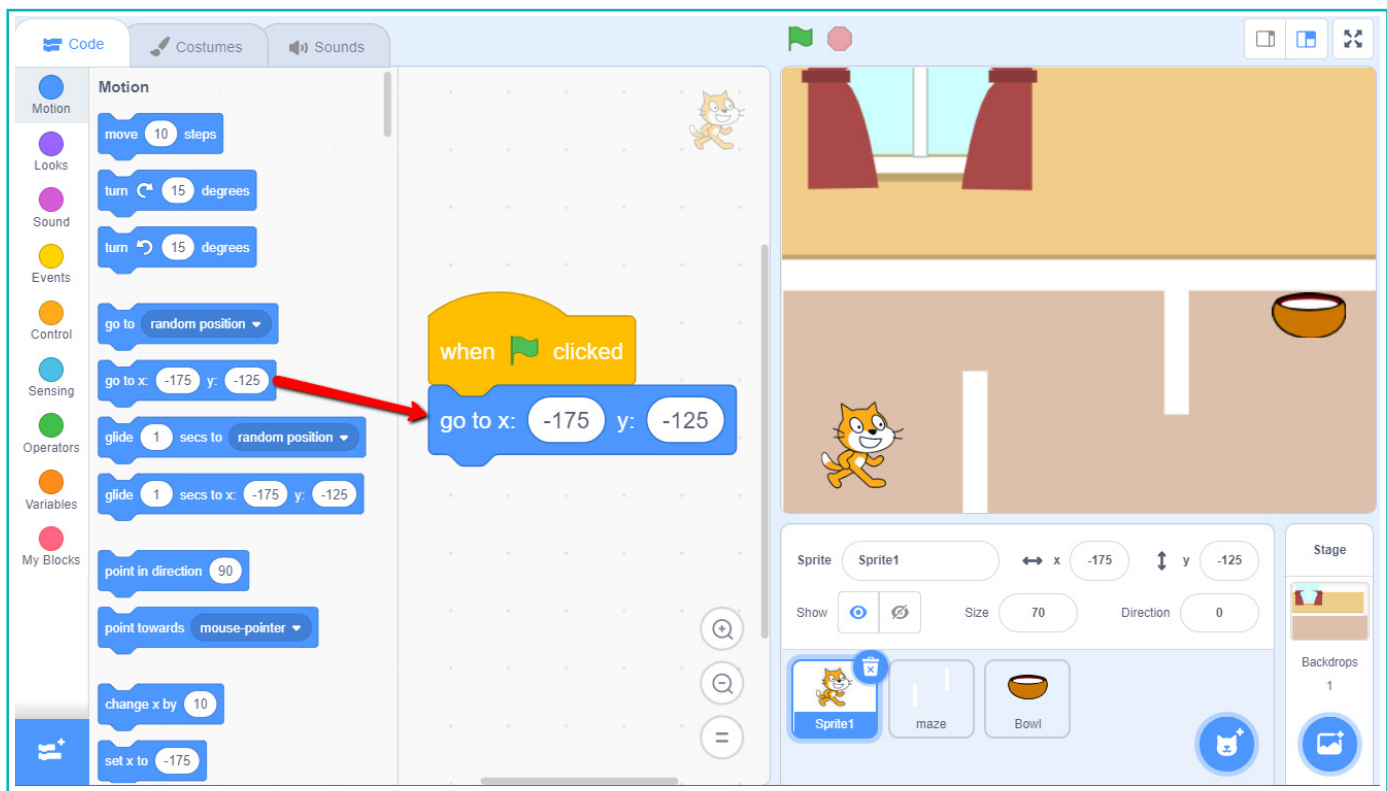


Steps

To create a code for the game, we will use scratch blocks.

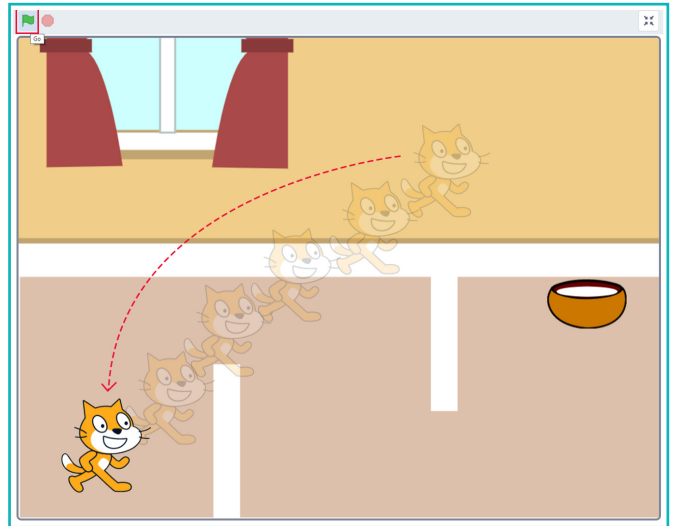
1 Set the start position for Tom.

- Our game starts when we click the flag
- To set Tom's position to X:-175, Y: -125, we will use the go to block



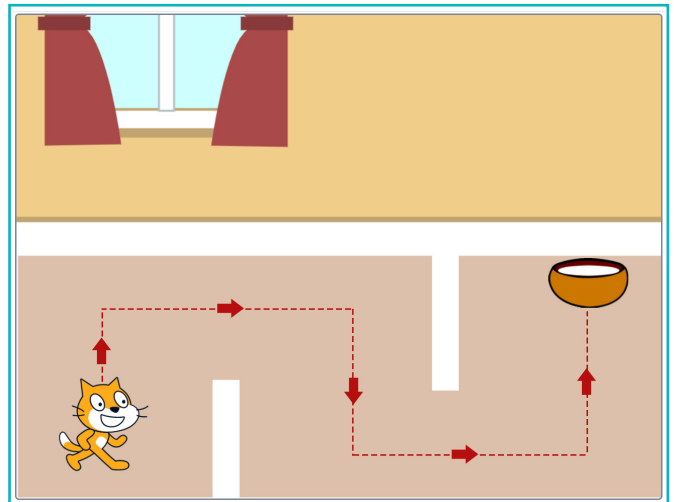
To check whether this works, we may move Tom to any position on the stage and click on the green flag.

For example: We have moved the cat near the wall. As we click on the green flag, it jumps to the prefixed position (-175, -125).

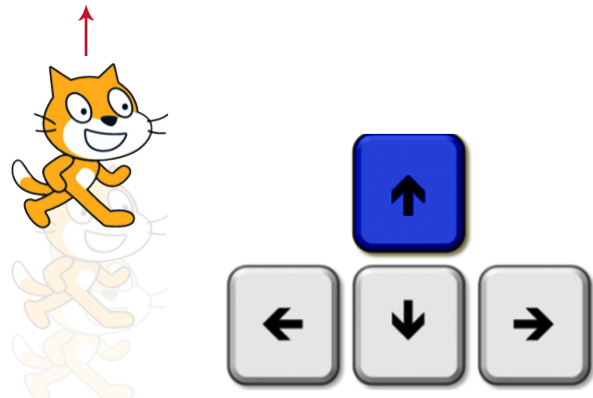


2 Give movements to Tom

The path to the bowl is not a straight one. To reach the milk bowl, Tom must move up, down, left, and right.



To play the game, the player must control Tom's movements. To control Tom, we will set keyboard movements. The up-arrow key will move Tom up and the down-arrow key will move him down. The left and right arrow keys will work similarly.



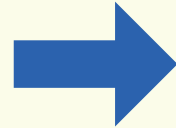
5. Conditions



Recall:



point in direction 90



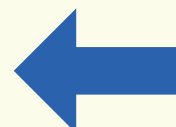
point in direction 0



point in direction 180

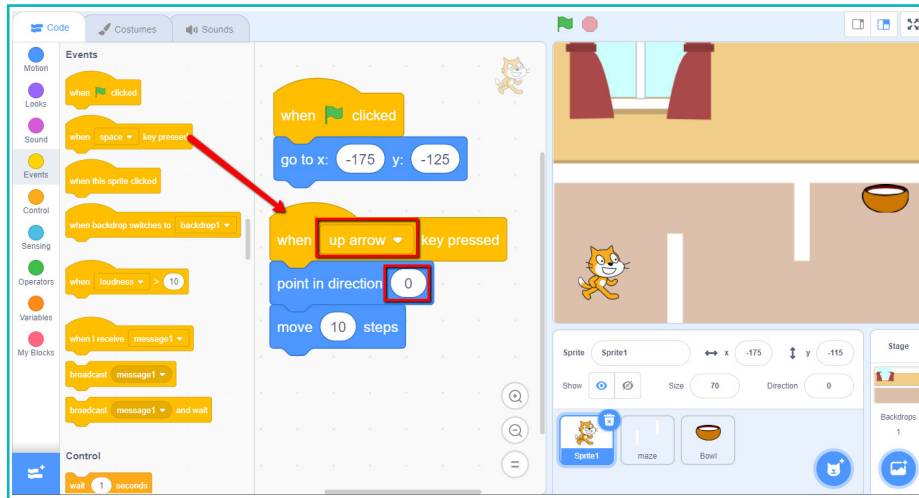


point in direction -90

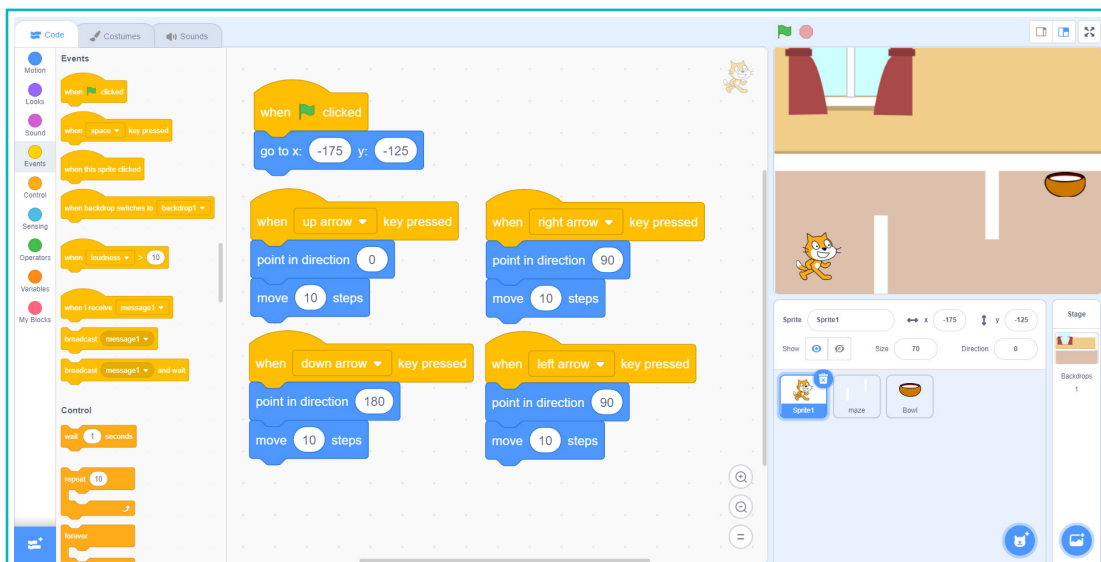
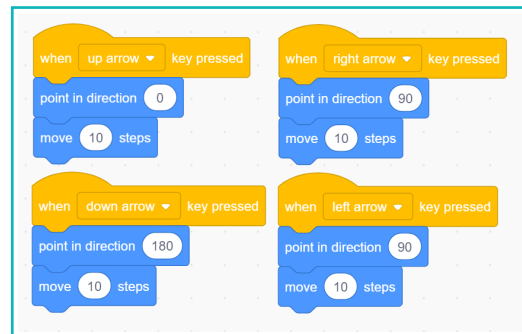
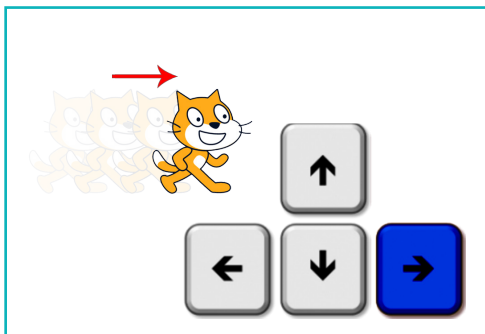


5. Conditions

To move Tom upward, we will add the following code:



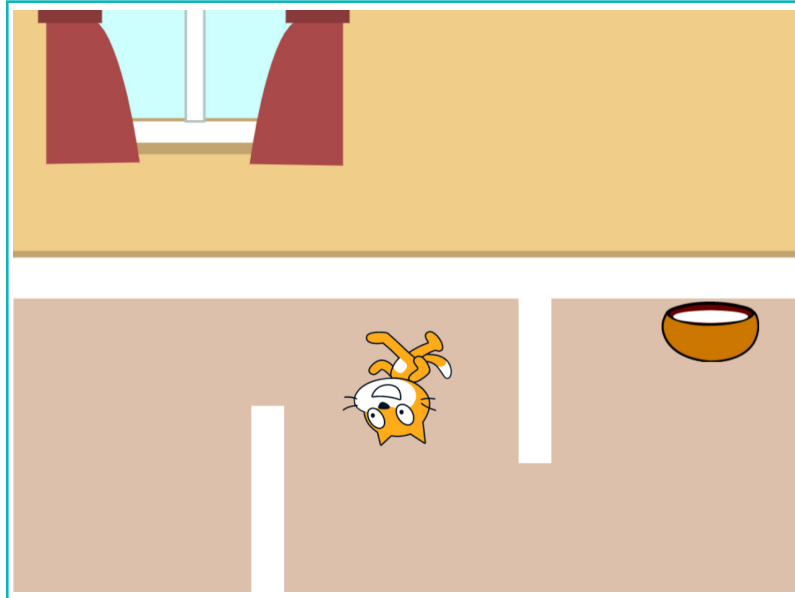
Set similar movements for all the directions.



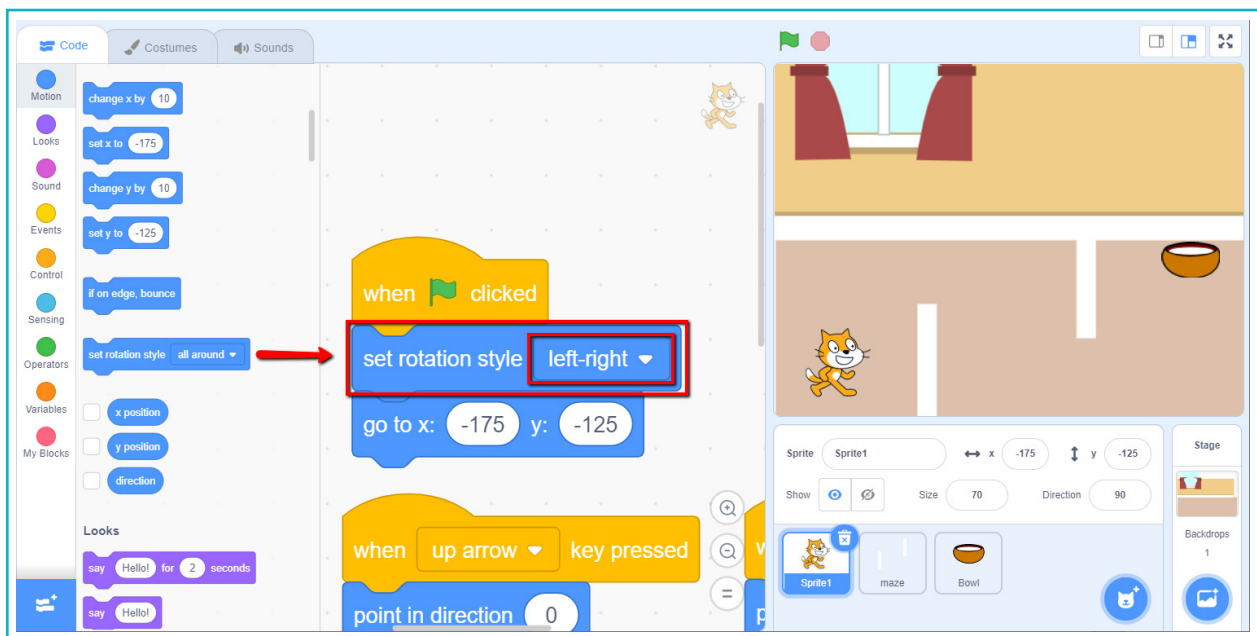
To test the movements, click on the green flag

5. Conditions

Notice that the sprite flips when we move it.



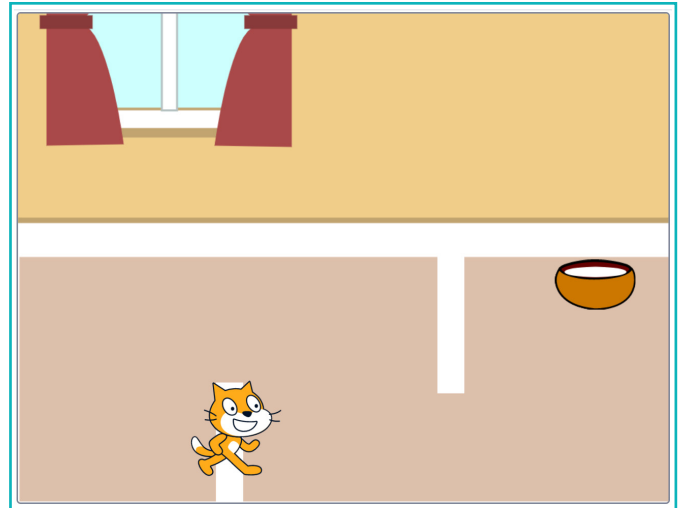
This is not how we want Tom to move. We do not want him to move upside down. To fix this, we will add a block to correct the rotation.



Test the movements once again.

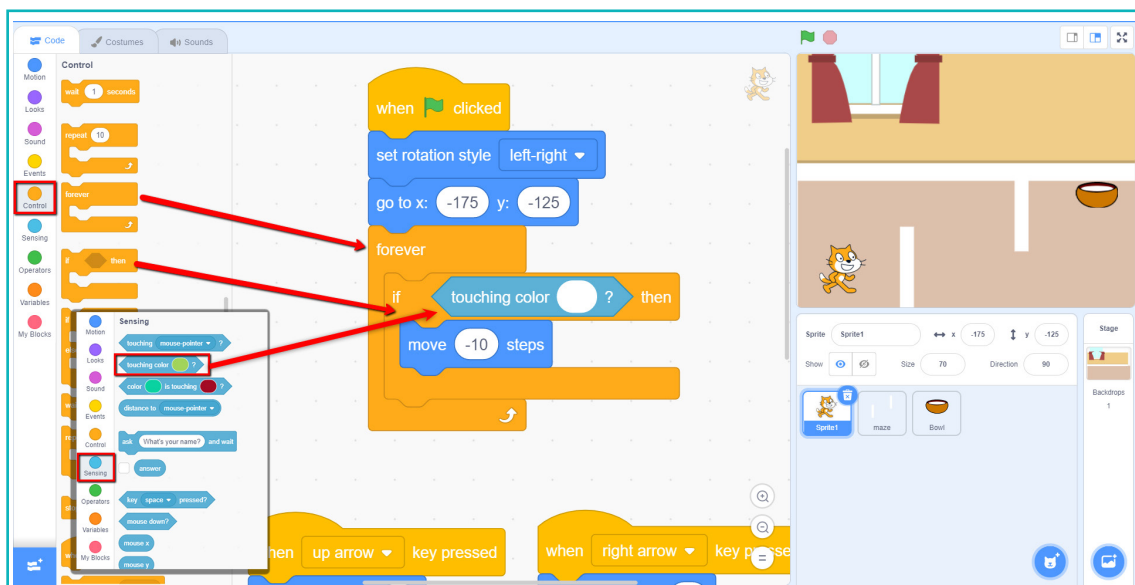
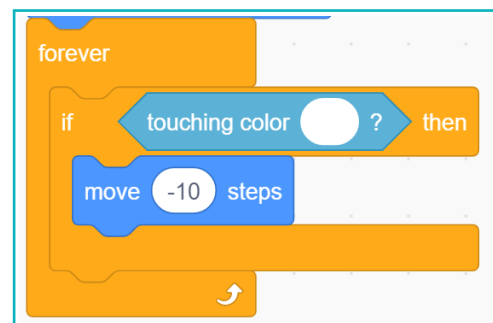
3 Make a real hurdle

Tom is passing through the hurdle. To fix this, we will add a code that moves the cat backward when he touches the white hurdle. The “**Sensing**” blocks enable a script so we can see what’s happening on the computer.



This block can sense when a sprite touches an area of a specific colour. So, if the sprite touches the colour white, we want to reverse the movement. We will add the “**if**” block inside the “**forever**” block. As we want the sprite to detect collision throughout the game, we use the forever block

Forever () : This is a **control block** . Blocks held inside this will loop forever.



Exercise

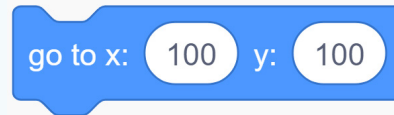
Match the pairs

4 a. To move forward 10 steps

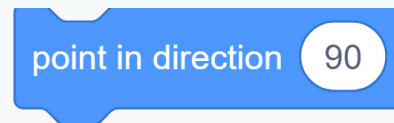
b. To set position at 100, 100

c. To rotate 90 degrees

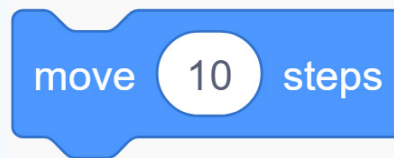
d.



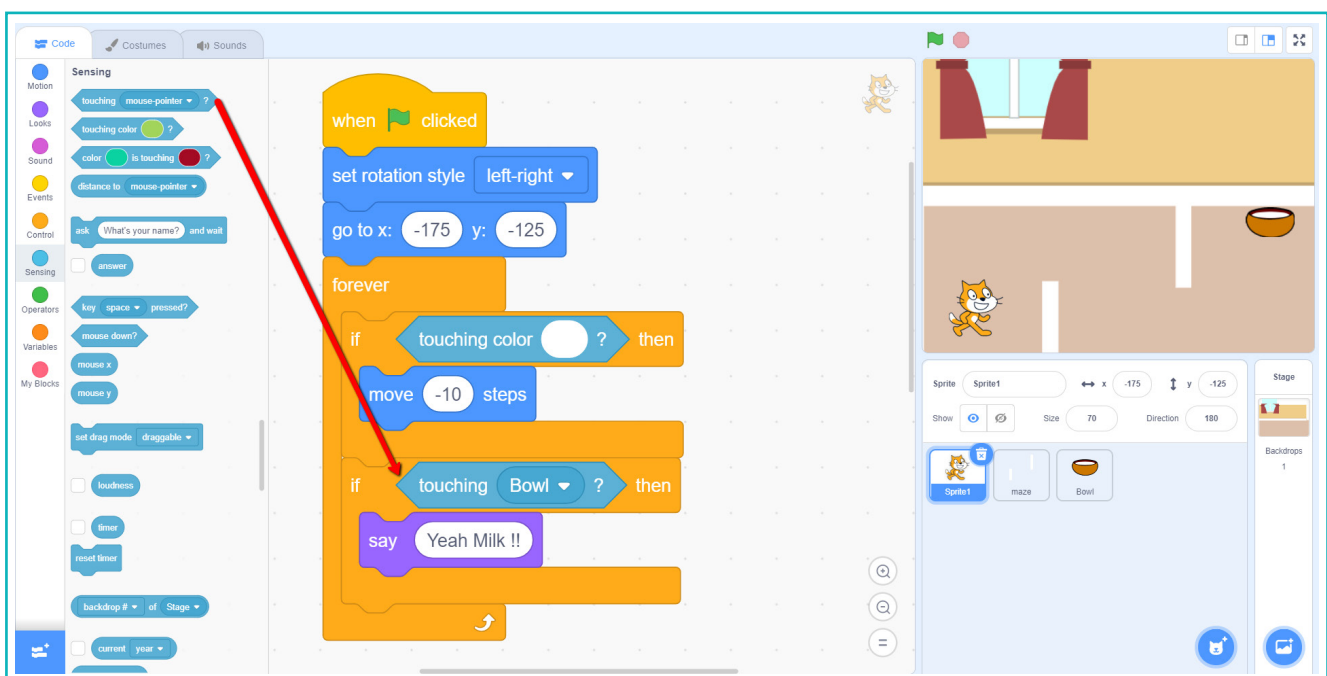
e.



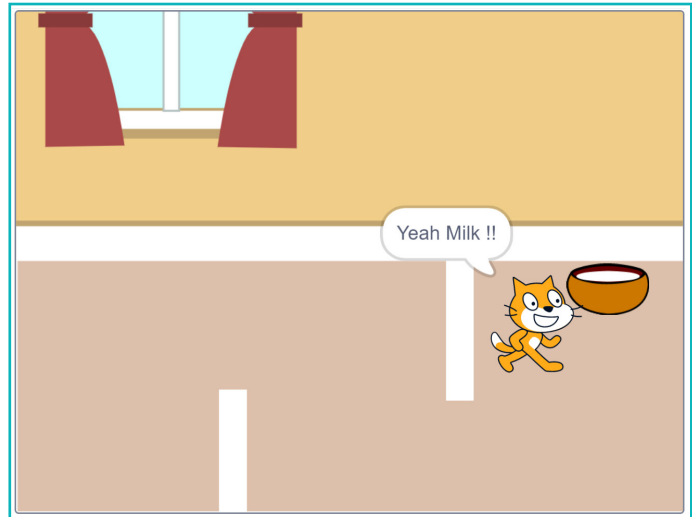
f.



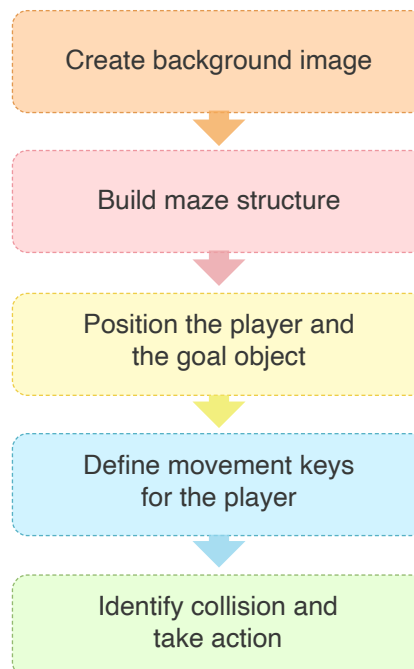
When Tom reaches the milk bowl, the game is over. We will display a message to show that the player has won.



Test the game once again.



Relook at the approach we used to create the game:



You can now create different structures, fix the position of the sprite, and use the “Forever” and “If...then” blocks.

Exercise

Tick Mark ☒ the correct answer

5 Which category of blocks is used to make sprites interact with their surroundings?

- a. Motion ☐ b. Looks ☐ c. Sensing ☐

6 Which block do we use to direct the sprite in a specific direction?

- a. Point in direction ☐ b. move ☐ c. if condition ☐

7 What is the category of “If () then” block?

- a. Sensing ☐ b. Control ☐ c. Motion ☐

Bytes

Blocks in scratch are divided into the following categories for easy access

Category	Notes
	Motion• Moves sprites, changes angles and position
	Looks• Controls the visuals of the sprite
	Sound• Plays audio files and effects
	Events• Event handlers
	Control• Conditionals and loops etc.
	Sensing• Sprites can interact with the surroundings
	Operations• Mathematical operators, comparisons
	Variables• Variable and list usage and assignment
	My Blocks• Custom Procedures



Tech Flash

If () Then block : This block checks if the condition is true. If it is, the blocks held inside it will run

Sensing category : Sensing blocks let the sprite interact with its surroundings.

Repeat () : This is a **control block**. Blocks inside this will loop a fixed number of times.

Forever () : This is a **control block**. Blocks held inside this will loop forever